Tabla de contenidos

1. *Networking* con Python
    a. `Sockets`
        i. Comandos básicos de sockets
            1. Cliente TCP
            2. Servidor TCP
            3. Ejemplo base

Networking con Python

En este material, veremos cómo implementar programas de **bajo nivel** en Python para que puedan comunicarse con otro programa, el cual puede encontrarse en el mismo computador o en otro.

Sockets

Un ***socket*** es un objeto del sistema operativo que permite a un programa transmitir y recibir datos desde y hacia otro programa corriendo en otra máquina, o en la misma máquina pero en otro puerto. Son nuestro "punto de entrada y salida a/desde la red" en nuestro código. Para obtener un *socket*, necesitamos (1) especificar qué tipo de dirección IP usaremos (IPv4 o IPv6), y qué protocolo de transporte usaremos (TCP ó UDP).

En Python, los *sockets* son provistos por el módulo `socket`. Para crear un *socket*, debemos crear una instancia de la clase `socket(family, type)`, donde `family` es el tipo de dirección IP que usaremos, y `type` es el protocolo de transporte. Algunos de los valores permitidos son:

- `family`
  - `AF_INET` para direcciones IPv4
  - `AF_INET6` para direcciones IPv6
- `type`
  - `SOCK_STREAM` para TCP
  - `SOCK_DGRAM` para UDP

Por ejemplo, para crear un *socket* TCP con IPv4, lo hacemos de la siguiente manera.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

print(type(sock))
```

```
<class 'socket.socket'>
```

Comandos básicos de sockets

Veamos cómo implementar una arquitectura cliente-servidor, utilizando *sockets*. Para poder implementarlo, utilizaremos varios conceptos del notebook "2-networking_conceptos_básicos" de la semana anterior, te recomendamos revisarlo si no recuerdas la materia.

Para poder comunicarnos utilizando *sockets* será necesario seguir una serie de pasos definidos para establecer la comunicación entre el cliente y el servidor, y asegurar que se envíen y reciban correctamente los mensajes.

Los pasos que se deben seguir se explican en la siguiente imagen:

Fuente: **HowDev:** Socket programming in Python

> **NOTA:** Por simplicidad, en el siguiente notebook solo enseñaremos cómo implementar un comunicación utilizando el protocolo TCP. Si quieres aprender cómo se utiliza el protocolo UDP, revisa el notebook bonus de esta semana.

Cliente TCP

Para establecer una conexión TCP necesitamos (1) la dirección IP del *host* destinatario, y (2) el puerto del *host* destinatario. Para conectarse a un servidor TCP usamos el método `connect((host, port))` del *socket*. El método recibe una tupla, cuyo primer elemento debe ser una dirección IP acorde a lo que especificamos al momento de crear el *socket*, y el segundo elemento es el puerto al que nos queremos conectar.

Opcionalmente, el primer elemento de la tupla puede ser un *string* con el *hostname*, como por ejemplo, `python.org`, `github.com` o `canvas.uc.cl`. En ese caso, el sistema utilizará el protocolo DNS para obtener la dirección IP correspondiente a ese *hostname*.

Por ejemplo, hagamos un cliente que se conecte a `104.18.32.7` en el puerto 80 vía TCP.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(('104.18.32.7', 80))
```

Si nos queremos conectar a stackoverflow.com también lo podemos hacer sin conocer su dirección IP.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.connect(('stackoverflow.com', 80))
```

Llegó la hora de enviar algún mensaje. Los *sockets* sólo transmiten *bytes*, por lo que cualquier cosa que enviemos debemos codificarla. Para enviar los *bytes* de un mensaje, se puede ocupar uno de los siguientes métodos de socket:

- `send(bytes)`: Trata de enviar el mensaje almacenado en la variable *bytes*. Sin embargo, puede que envíe solo parte de este en vez del mensaje completo, o el envío resulte en error. Debido a lo anterior, este método retorna la cantidad de *bytes* que fue efectivamente enviada, dejando como responsabilidad al programador el llamar nuevamente `send` para enviar los *bytes* restantes hasta que se haya enviado el mensaje completo. Nos daremos cuenta de esto cuando la suma de la cantidad de bytes enviados en cada llamada al método sea igual al largo del mensaje original.
- `sendall(bytes)`: Este método se asegura de enviar todos los *bytes* del mensaje, haciendo uso por debajo del método `send` hasta que el mensaje se haya enviado en su totalidad. Sin embargo, en caso de haber un error en el envío, este método solo lanzará una excepción, por lo que no podremos saber cuántos bytes efectivamente se habían enviado de forma previa a que ocurriera el error.

En el siguiente ejemplo, los *bytes* contienen un mensaje en el formato HTTP, y corresponde a una **solicitud HTTP** (*HTTP Request*) que puede ser comprendida por un servidor web. Los enviaremos mediante el método `sendall`:

```
solicitud = 'GET / HTTP/1.1\nHost: stackoverflow.com\n\n'
sock.sendall(solicitud.encode('ascii'))
```

¿Qué nos habrá respondido el servidor? Para saberlo, debemos ocupar el método `recv(buffer)`, que nos retornará un objeto `bytes`. El parámetro que recibe este método es la cantidad **máxima*** de *bytes* que será leída. La documentación sugiere colocar una potencia de 2 pequeña, como 4096.

*Es la cantidad máxima, porque lo que hace es sacar los bytes que hayan llegado al socket en ese instante. Si es que en el socket habían llegado 20 bytes al momento de llamar a `sock.recv(4096)`, entonces el método me entregará solo 20 bytes.

```
data = sock.recv(4096)
print(data.decode('ascii'))
```

```
HTTP/1.1 301 Moved Permanently
Date: Thu, 30 Apr 2026 13:05:49 GMT
Content-Length: 0
Connection: keep-alive
Location: https://stackoverflow.com/
set-cookie: __cf_bm=5gEAIOlyavrWEu6XJQYROZnp38MjpDH8G2OTr3tzucs-
1777554349.3086414-1.0.1.1-
UEHbgCMKiv0_4Auo.0UUONgRiJP0ls2H3_axSViTMxxEZmykZehdpph.fuxrAxMishYoS9
9UE.Hy80xPAlTv82M2FLcpnJ7FYftRTtGCqMSwflY90Sp.OsinYwnT3iRI; HttpOnly;
Path=/; Domain=stackoverflow.com; Expires=Thu, 30 Apr 2026 13:35:49
GMT
X-DNS-Prefetch-Control: off
Server: cloudflare
CF-RAY: 9f46c29b29def3b5-SCL
```

El último paso, pero no el menos importante: **debemos cerrar la conexión**; de lo contrario, si nuestro programa continúa, estará ocupando recursos que no utiliza.

```
sock.close()
```

Hagamos el ejercicio completo en una sola celda. Además, aprovechemos de manejar las excepciones pues, en *networking*, es muy probable que se produzcan errores. La clase `ConnectionError` es la clase base para distintos errores relacionados con problemas de conexión, por lo que capturando su excepción podremos responder a cualquier problema que ocurra en este sentido.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

host = "stackoverflow.com"
port = 80

try:
    sock.connect((host, port))
    msg = 'GET / HTTP/1.1\nHost: stackoverflow.com\n\n'
    sock.sendall(msg.encode('utf-8'))
    data = sock.recv(4096)
    print(data.decode('utf-8'))
except ConnectionError as e:
    # ConnectionError es la clase base de BrokenPipeError,
ConnectionAbortedError,
    # ConnectionRefusedError y ConnectionResetError
    print("Ocurrió un error.")
finally:
    # ¡No olvidemos cerrar la conexión!
    sock.close()

HTTP/1.1 301 Moved Permanently
Date: Sat, 27 Jun 2026 13:37:23 GMT
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
Connection: keep-alive
Location: https://stackoverflow.com/
set-cookie: __cf_bm=izcBkthel7F6OSNI2HYhFRoUtrDJUub5Lv8WJKtlzLw-
1782567443.447684-1.0.1.1-
tTyIoUVsmMAsiMy0KYw0pk8Ft7Cx_1fK3AE1HpBLcBIwpm.xARvKw6iO50LwBIDiOav0mV
bhJe415PLiIdeMmvzFf4i3OawqrWBACGiXZX6fsF_umE73jruV9aJkHrxLPvJJ1DDzrPHm
a90yP14RFAMzXFeaO9F8WcRMxQKNutI; HttpOnly; Path=/;
Domain=stackoverflow.com; Expires=Sat, 27 Jun 2026 14:07:23 GMT
X-DNS-Prefetch-Control: off
Server: cloudflare
CF-RAY: a124d8998e53fa3d-SCL

a7
```

```
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>cloudflare</center>
</body>
</html>

0
```

La respuesta que obtenemos del servidor web ubicado en el puerto 80 de `stackoverflow.com` es una **respuesta HTTP** (*HTTP Response*) que contiene a lo más 4096 *bytes*. La respuesta HTTP es una de 301 (moved permanently), donde se indica que para conectarse a StackOverflow hay que usar la nueva dirección indicada, usando HTTPS.

¿Qué significa esto? El puerto 80 es usado por los servidores. Se trata un servicio de **texto plano**, esto es, sin encriptación. La mayoría de los servidores web modernos utilizan una versión de HTTP con encriptación, conocida como HTTPS y que funciona en el puerto 443. Conectarse a un servidor web con HTTPS requiere operaciones de encriptación sobre el *socket* que complicarían innecesariamente este ejemplo.

Servidor TCP

> **NOTA**: Debido a la naturaleza de este contenido, esta sección contiene porciones de código separados que **no pueden ser ejecutados** con el fin de poder explicarlas de forma ordenada, ya que no es recomendable ejecutarlos en Jupyter Notebook. Si quieres ejecutarlo, en la carpeta `codigo/TCP` encontrarás un código similar al que veremos a continuación.

Un servidor debe escuchar las conexiones que le llegan a él en un puerto específico. Para ello, debemos "enlazar" o "asociar" un *socket* a ese puerto con el método `bind((host, port))`. El primer elemento de la tupla es el *hostname* del *host* en que estamos corriendo el servidor; el segundo elemento es el puerto donde queremos escuchar las conexiones.

Supongamos que queremos escuchar en el puerto 9000.

```
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Poner como host 'localhost' indica que el host será nuestro
computador
host = 'localhost'
port = 8998

sock.bind((host, port))
```

En el cliente no es necesario hacer un *bind* entre el *host* y el puerto, ya que el sistema operativo lo hace implícitamente a través del método `connect`, asignando al cliente un puerto aleatorio.

Si el servidor al cual nos conectamos exigiera que el puerto de cada cliente esté en un rango específico, tendríamos que "enlazar" el cliente a un puerto específico también.

Ahora, empecemos a escuchar conexiones con el método `listen(conexiones_pendientes)`. El parámetro `conexiones_pendientes` indica cuántas conexiones **pendientes (sin aceptar)** pueden estar de forma paralela esperando a ser aceptadas por el servidor, antes de que este rechace cualquier conexión nueva sobre ese máximo.

```
sock.listen(3) # Como máximo pueden haber hasta 3 clientes esperando ser aceptados al mismo tiempo.
```

Ahora, empecemos a aceptar conexiones. Para ello, ocupamos el método `accept()`, que nos retornará un *socket* con el que podremos comunicarnos con ese cliente en específico, y la dirección de ese cliente (en específico, una tupla con su dirección IP y su puerto). El método esperará hasta que haya una conexión entrante; por lo tanto, si nadie se conecta, el programa quedará congelado.

```
while True:
    socket_cliente, address = sock.accept()
    print("Conexión aceptada desde", address)
    socket_cliente.sendall("Gracias por conectarte\n".encode("utf-8"))
    socket_cliente.close()
```

Cuando queramos terminar el programa servidor, debemos cerrar el *socket* que estaba aceptando las conexiones.

```
sock.close()
```

Ejemplo base

Hagamos el ejercicio completo, ejecuta los códigos y mira cómo se comportan. Puedes encontrar los códigos del servidor y el cliente en la carpeta `codigo/TCP`. Para ejecutarlos, debes abrir múltiples terminales, en una debes ejecutar el archivo del servidor y en las otras el archivo del cliente.

Primero revisemos el código del servidor. Este se comporta igual que el que revisamos anteriormente: recibe a un cliente, le manda un mensaje y cierra la conexión. Adicionalmente, este servidor presenta una singularidad respecto al otro código, ya que el servidor cerrará después de aceptar cinco clientes.

```
## Código servidor

import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Poner como host 'localhost' indica que el host será nuestro computador
```

```
host = 'localhost'
port = 8998

sock.bind((host, port))
sock.listen()
print("[Servidor] Aceptando conexiones en el puerto", port)

counter = 0
while counter < 5:
    try:
        socket_cliente, address = sock.accept()
        print("[Servidor] Conexión aceptada desde", address)
        socket_cliente.sendall("Gracias por conectarte\n".encode("utf-
8"))
        socket_cliente.close()
        counter += 1
    except ConnectionError:
        print("[Servidor] Ocurrió un error.")

print("[Servidor] Se han recibido 5 clientes, por lo que se cerrará la
conexión")
sock.close()
```

En cuanto al cliente, este intentará conectarse al servidor y recibir el mensaje que envíe. Una vez recibido el mensaje, o si sucede un error, el cliente cerrará la conexión con el servidor y se terminará el programa.

```
## Código cliente

import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Poner como host 'localhost' indica que el host será nuestro
computador
host = 'localhost'
port = 8998

try:
    sock.connect((host, port))
    data = sock.recv(4096)
    print(data.decode("utf-8"))
except ConnectionError as e:
    print("[Cliente] Ocurrió un error")
finally:
    sock.close()
```